

Design and Engineering of Computing Systems

CS-744 Project Report

Ashutosh Birla

Roll Number: 24m0752

Professor: Mythili Vutukuru

November 24, 2025

Contents

1	System Architecture and Workflow	3
1.1	Core Components	3
1.2	Workflow and Processing Paths	3
2	Development Process	4
2.1	Implementation Details	4
3	Load Generator Design	4
3.1	Architecture	4
3.2	Optimization for High Stress	5
4	Experimental Setup	5
4.1	Resource Isolation (Core Pinning)	5
4.2	Methodology	5
5	Performance Evaluation	5
5.1	Workload 1: Write-Intensive (Put All)	5
5.2	Workload 2: Read-Intensive (Get Popular)	6
6	Conclusion	7

1 System Architecture and Workflow

The objective of this project was to design, engineer, and evaluate a high-performance, multi-tier Key-Value store system. The system comprises three distinct components designed to handle concurrent client requests with varying performance characteristics.

1.1 Core Components

- **C++ Server:** A multi-threaded HTTP server that handles incoming client requests. It implements the core application logic and orchestrates data flow between the cache and the database.
- **FIFO Cache:** An in-memory storage mechanism embedded within the server. It uses a First-In-First-Out (FIFO) eviction policy to manage data, providing rapid retrieval for frequently accessed keys (Hot Data).
- **MySQL Database:** A persistent storage backend that ensures data durability. It is accessed via the `mysql-connector-c++` library.

1.2 Workflow and Processing Paths

The system handles two primary types of requests, each following a distinct processing path with different performance characteristics:

1. **Read Path (GET Requests - Fast Path):** Incoming read requests are first checked against the in-memory FIFO Cache.
 - *Cache Hit:* If the data is found, it is returned immediately. This path is CPU-bound as it involves only memory lookups and thread scheduling.
 - *Cache Miss:* If data is missing, the server fetches it from the MySQL database, updates the cache, and returns the response.
2. **Write Path (PUT/POST Requests - Slow Path):** Write requests bypass the read cache to ensure strict consistency and durability. They are written synchronously to the MySQL database. This path is Disk I/O-bound due to the physical latency of committing transactions to storage.

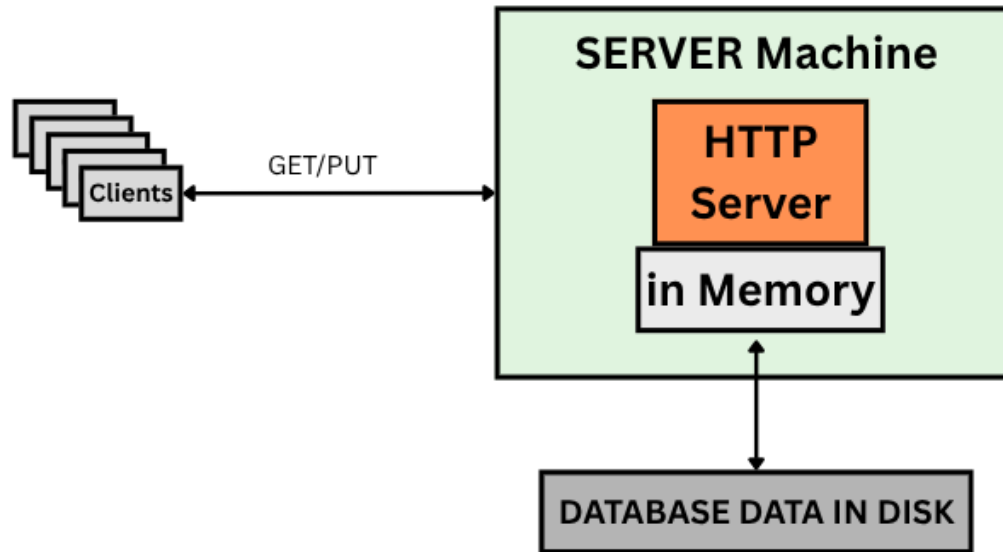


Figure 1: System Architecture

2 Development Process

The development of the system involved writing approximately 400 lines of C++ code, focusing on thread safety and performance optimization.

2.1 Implementation Details

- **Core Logic:** Implemented the multi-threaded server architecture and the thread-safe FIFO cache logic manually.
- **Third-Party Integration:**
 - `cpp-httpplib`: Used for robust HTTP networking and request handling.
 - `mysql-connector-c++`: Integrated for reliable database operations.
- **Build System:** The project is built using CMake. Configuration for the build system and the SQL connector integration was achieved by adapting standard documentation and technical references available online.

3 Load Generator Design

To rigorously evaluate the system, a custom closed-loop load generator was developed. Unlike open-loop generators that fire requests blindly, this tool is designed to respect the system's capacity while applying maximum stress.

3.1 Architecture

- **Synchronous Multi-Threading:** The client uses `cpp-httpplib` in a synchronous manner. Each simulated user is a separate thread that blocks while waiting for a response.

- **Closed-Loop Model:** A new request is triggered *only* upon the completion of the previous request.

3.2 Optimization for High Stress

To ensure the load generator itself did not become the bottleneck:

- **Dedicated Resources:** The load generator was assigned dedicated CPU cores (using `taskset`) to prevent resource contention with the server.
- **Code Optimization:** High-latency operations, such as printing to the console (`cout`), were removed from the critical path. Thread-safe random number generators were used to avoid mutex contention during workload generation.

4 Experimental Setup

The load tests were conducted on a multi-core Linux environment with strict resource isolation to identify hardware bottlenecks accurately.

4.1 Resource Isolation (Core Pinning)

Process affinity was enforced using `taskset` to isolate components:

- **Core 0:** MySQL Database (Handling Disk I/O and Transactions).
- **Core 1:** C++ Server (Handling Application Logic and Cache).
- **Cores 2-3:** Load Generator (Generating Traffic).

4.2 Methodology

- **Workload Profiles:** Two distinct scenarios were evaluated:
 1. *Put All:* A write-intensive workload designed to stress the database disk I/O.
 2. *Get Popular:* A read-intensive workload where a small set of keys is requested repeatedly, ensuring a high cache hit rate to stress the CPU.
- **Scalability Testing:** The number of client threads was incremented (8, 16, 32, 64, 128) to find the saturation point.
- **Metrics:** Throughput (Requests/Sec) and Response Time (Latency) were measured at the client side. Hardware usage was monitored using `mpstat` (CPU) and `iostat` (Disk).

5 Performance Evaluation

5.1 Workload 1: Write-Intensive (Put All)

This workload forces the system to interact with the persistent storage for every request, bypassing the cache.

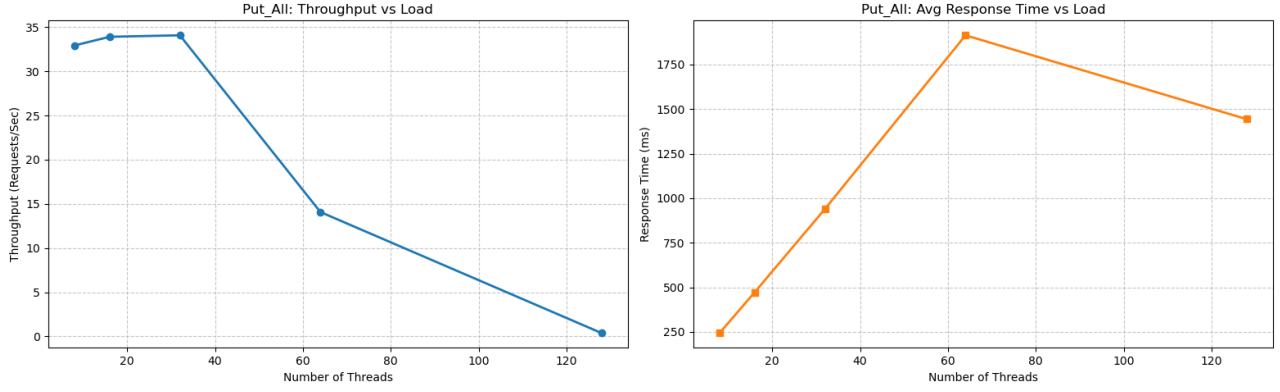


Figure 2: Put All Metrics

Analysis:

- **Throughput:** As shown in Figure 2, throughput drops significantly as the number of threads increases (from ≈ 35 req/sec down to < 5 req/sec at 128 threads).
- **Latency:** Response time spikes dramatically, peaking at over 1900ms.
- **Bottleneck:** The system is **Disk I/O Bound**. The database cannot commit transactions fast enough to keep up with the incoming requests, causing threads to queue up and wait for disk operations.

5.2 Workload 2: Read-Intensive (Get Popular)

This workload requests a small set of keys repeatedly, ensuring a near 100% cache hit rate.

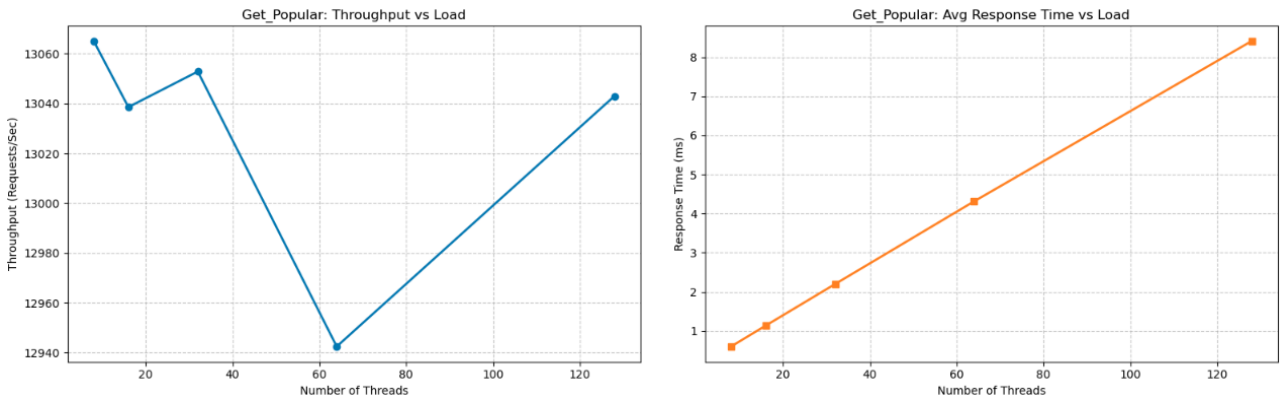


Figure 3: Get Popular Metrics

Analysis:

- **Throughput:** The throughput remains stable and high ($\approx 13,000$ req/sec) regardless of the thread count variation shown.
- **Latency:** Response time increases linearly with load but remains very low (0.5ms to 8ms).

- **Bottleneck:** The system is **CPU Bound**. Since data is served from RAM, the disk is idle. The limit is determined by how fast the CPU can process network packets, manage threads, and perform memory lookups.

6 Conclusion

The project successfully demonstrated the performance characteristics of a multi-tier system. By isolating resources and using a custom closed-loop load generator, we identified clear bottlenecks:

1. **Disk I/O Saturation** occurred during write-intensive workloads, severely limiting throughput.
2. **CPU Saturation** occurred during cached read-intensive workloads, allowing for high throughput with low latency.